Stress-based Graph Layout with Particle Swarm Optimization

Marko Cvijetinović

May 11, 2026

1 Introduction

Stress-based Graph Layout is a technique that aims to create readable images of graphs. The goal is to match Euclidean distances in the planar representation as closely as possible to the shortest-path distances in the graph. It does so by minimizing the stress function

$$\text{Stress}(X) = \sum_{i<j} w_{ij}(\|x_i - x_j\| - d_{ij})^2$$

where x_i and x_j are the 2D positions of the nodes i and j in the layout, d_{ij} is the shortest-path distance between the nodes and w_{ij} is usually $1/d_{ij}^2$ to ensure equal treatment of distances of varying magnitudes. In this paper, Particle Swarm Optimization (PSO) will be used for this challenge, which is an optimization technique inspired by collective social behavior of animals, such as pigeons or fish. Applications of Particle Swarm Optimization to stress-based graph layout appear to be significantly less common in the literature than specialized numerical optimization approaches such as Kamada-Kawai.

2 Implementation

2.1 Graph Layout

To define optimization goal, the most important functions are to calculate path distances in the graph and Euclidean distances in its layout. The former uses Floyd Warshall algorithm from NetworkX python library with minor modifications, whereas the latter is integrated into the final stress computing function that goes over all pairs of nodes. An important decision was to normalize graph paths, which effectively made graphs of varying sizes more "similar", resulting in PSO parameter (cognitive, inertial and social learning speeds) stability across most graphs. Instead of using Python's for loops, the stress computing function is optimized to use NumPy's vectors, significantly improving the computation speed.

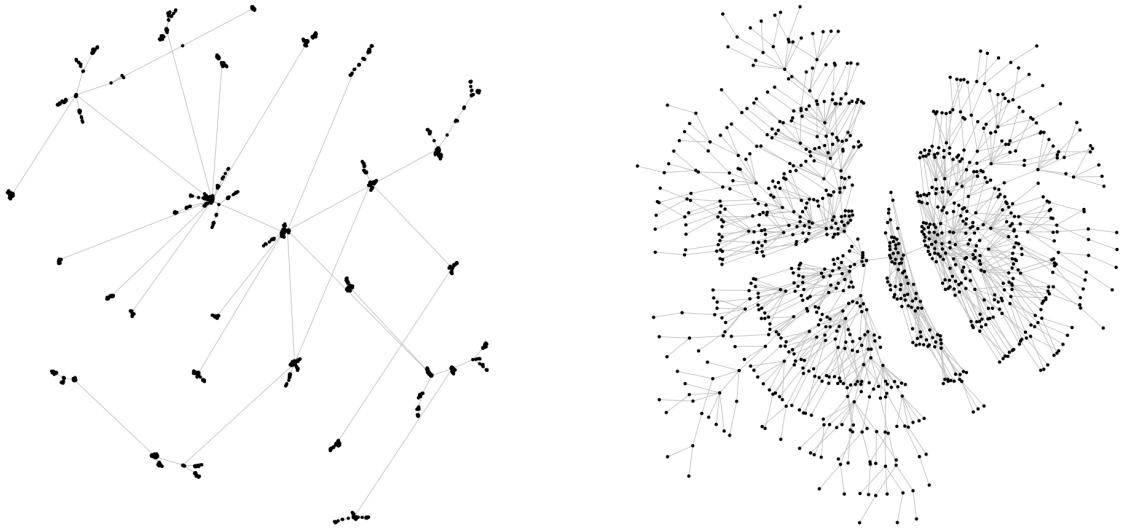


Figure 1: Example layouts by David Schoch

2.2 Particle Swarm Optimizer (PSO)

PSO was separated into a completely abstract implementation and a module that adapts it for stress-based graph layout problem. Abstract PSO features a classic implementation of the algorithm, updating speed and position in each iteration using inertia, cognitive and social coefficient parameters. For particle i:

$$v_i = c_{in} * v_i + c_{co} * r_{co} * (x_i^{best} - x_i) + c_{so} * r_{so} * (x^{swarm} - x_i)$$

$$x_i = v_i + x_i$$

where r_{co} and r_{so} are random weights chosen in each evaluation, v_i is particle's speed, and x_i, x_i^{best} and x^{swarm} are particle's current position, best personal recorded position and swarm's best recorded position. Personal best updates if fitness function of new position is lower than fitness function of personal best. Swarm's best updates if fitness function of new personal best is lower than fitness function of swarm's best.

The main unique aspect of this implementation is generalizing fitness function, initialize values and clip to boundary steps, to allow any fitness goal, custom starting points and arbitrary repair mechanisms such as clip position, clip velocity, mutation or normalization. Optionally, a callback function can be passed, it will trigger at each iteration, which may be useful for printing results or drawing graphs. The problem specific PSO module's main component is a function that can generate all functions abstract PSO needs. Initial position for all particles is random, fitness function is the stress computing function discussed in the previous section and repair function offers a variety of modes. Most useful of them for this problem was normalizing the graph after each iteration, centering it again in (0, 0). For stress-layout problem, a graph's relative position is irrelevant, therefore its "drifting" in space is pure noise.

2.3 Batching

After normalizing the graph in various ways, which stabilized PSO parameters, the most obvious bottleneck was speed. The complexity of the entire algorithm is

$$O(n^3 + k * p * n^2)$$

where n is the number of nodes in a graph, n^3 is the complexity of Floyd Warshall algorithm, k is the number of PSO iterations, p is the number of PSO particles and n^2 is the complexity of stress computing function used for fitness, which makes initialize and repair functions, both with linear complexities, negligible. Compute stress run for each particle in each iteration easily dominates over Floyd Warshall for any sensible parameter choice. While vectorizing the double loop of compute stress in NumPy was substantial, it was not enough to stop speed from being the absolute bottleneck. The solution was to instead batch the calculations of compute stress on GPU with PyTorch, allowing it to run parallel for each particle, leading to massive improvements. For this, both a new compute stress function and PSO algorithm had to be created. BatchedPSO splits the main "move" function of Particle into multiple steps, in order for fitness function that accepts all particles at once to be inserted between.

3 Experimental Results

3.1 Algorithms

In this chapter Batched PSO will be compared to a brute force algorithm and NetxworX's spring and Kamada Kawai layout. Since this is a continuous optimization problem, a standard brute force method to check everything is unfeasible, therefore a brute force algorithm that makes random layouts and saves the best one was implemented. Brute force approach was also batched on GPU for improved speed. Spring layout is an optimization method that models weights as repellent force, it only considers direct neighbors, instead of all paths in graphs like stress-based graph layout requests. That gives it linear complexity in terms of nodes (for most graphs), however it should be noted it is not minimizing exactly the same function as we want. Kamada Kawai layout applies iterative numerical optimization to the same stress function introduced at the start of this paper.

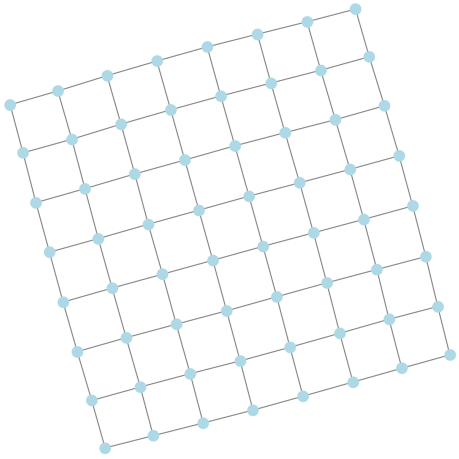
3.2 Parameters and Characteristics

Brute Force: $samples = 50000, batch\_size = 1000$
Sprint Layout: $iterations = 8000$
Batched PSO: $particle\_count = 400, iterations = 6000, c_{in} = 0.8, c_{so} = 1.4, c_{co} = 0.8$
CPU model: Intel(R) Core(TM) i5-10600K CPU @ 4.10GHz
GPU model: GB203 [GeForce RTX 5070 Ti]
OS: Ubuntu 24.04.4 LTS
Compiler: Python3

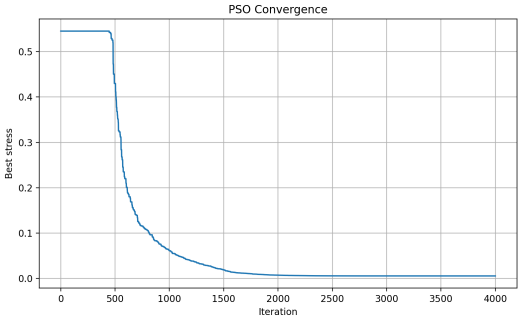
Algorithm Accuracy (Minimized Stress) and Speed				
	Brute Force	Spring Layout	Kamada-Kawai Layout	Batched PSO
Path 30	0.39 0.57s	0.0043 0.44s	0.0073 0.25s	0.00000006 67s
Cycle 30	0.34 0.21s	0.029 0.44s	0.031 0.002s	0.0024 67s
Grid 2D 8x8	0.39 0.23s	0.0067 1.1s	0.0078 0.03s	0.0055 74s
Connected Caveman 8x8	0.46 0.24s	0.026 1.1s	0.013 0.02s	0.0056 74s
Karate Club	0.21 0.21s	0.11 0.5s	0.068 0.03s	0.029 67s
Barbasi Albert 150x3	0.29 0.37s	0.12 4.8s	0.11 0.16s	0.11 89s
Balanced Tree 4x4	0.39 0.78s	0.14 24s	0.091 1.6s	0.082 133s

3.3 Results Interpretation

As expected, brute force algorithm did not perform well, and it shows how difficult this optimization problem is. Spring Layout did moderately well, however optimizing a different cost leads to its inability to reduce error below a certain threshold. Kamada Kawai produced excellent results in both accuracy and speed. While BatchedPSO has technically beaten all of them, the high computation speeds should be taken into account. Running PSO with as many particles and iterations as I did is likely unnecessary, however it is interesting to see how much the stress can be lowered when given "unlimited" time. Furthermore, a noticeable trend can be observed, PSO loses its advantage over Kamada-Kawai on larger graphs. I believe that is not necessarily because different learning speeds are required, but search space becomes so complex, same margins of error are no longer tolerated, resulting in particles getting stuck in local minima. While studying learning parameters might result in a more resilient algorithm, Kamada Kawai has the edge that it always works without parameter tuning.

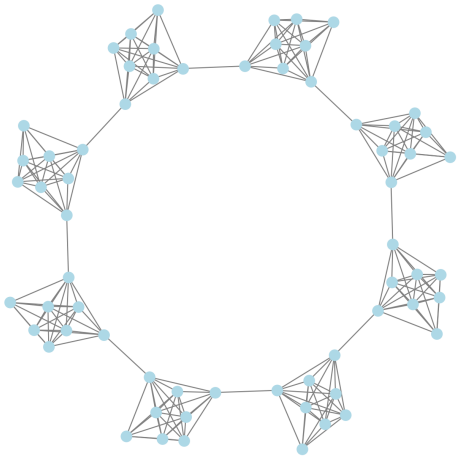


(a) Final layout

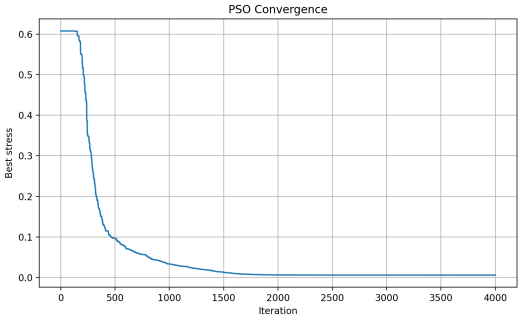


(b) Convergence graph (stress function over iterations)

Figure 2: Grid 2D Graph 8x8 PSO optimization



(a) Final layout



(b) Convergence graph (stress function over iterations)

Figure 3: Connected Caveman 2D graph 8x8 PSO optimization

4 Conclusion

PSO can reach stress values better than Kamada–Kawai, but only at substantially higher computational cost. Regardless, it may be useful for discovering general improvements for PSO. While parameters are stable across many graphs as previously stated, large graphs will still require some parameter tuning. Early or slow convergence starts at about 100 nodes and worsens as graphs become larger. As a possible way to fix parameter instability and manual tuning, I envision a PSO algorithm that has more than one global best value. Similar to how the population of pigeons may not necessary all follow the same lead, maybe a flock can be separated into a couple of sub-flocks where each pigeon gravitates towards the best value of its sub-flock using social velocity. For the exact number of sub-flocks, I would choose a square root of particle number. To avoid this being exactly the same as running PSO more than once, perhaps a weaker pool towards true global best can be added. Increasing mutation to free particles from local minima can be another approach. Some unrelated ideas for improving PSO for stress-based graph layout problem can be sampled or local stress for large graphs to make the complexity linear (in terms of nodes) and attempt to prove or disprove my hypothesis about stability of learning parameters for normalized graphs by studying them across graph families.

References

[1] J. Kennedy and R. Eberhart, *Particle Swarm Optimization*, Proceedings of IEEE International Conference on Neural Networks, 1995.

[2] T. Kamada and S. Kawai, *An Algorithm for Drawing General Undirected Graphs*, Information Processing Letters, vol. 31, no. 1, pp. 7–15, 1989.

[3] E. R. Gansner, Y. Koren and S. North, *Graph Drawing by Stress Majorization*, Proceedings of the 12th International Symposium on Graph Drawing, 2004.

[4] T. M. J. Fruchterman and E. M. Reingold, *Graph Drawing by Force-Directed Placement*, Software: Practice and Experience, vol. 21, no. 11, pp. 1129–1164, 1991.

[5] S. Kapunac, *Materials for Computational Intelligence classes*, 2025.